Vansh Singhal

Roll No: 2501010231

Subject: Nand2Tetris

Course: B.Tech CSE CORE

Section: C

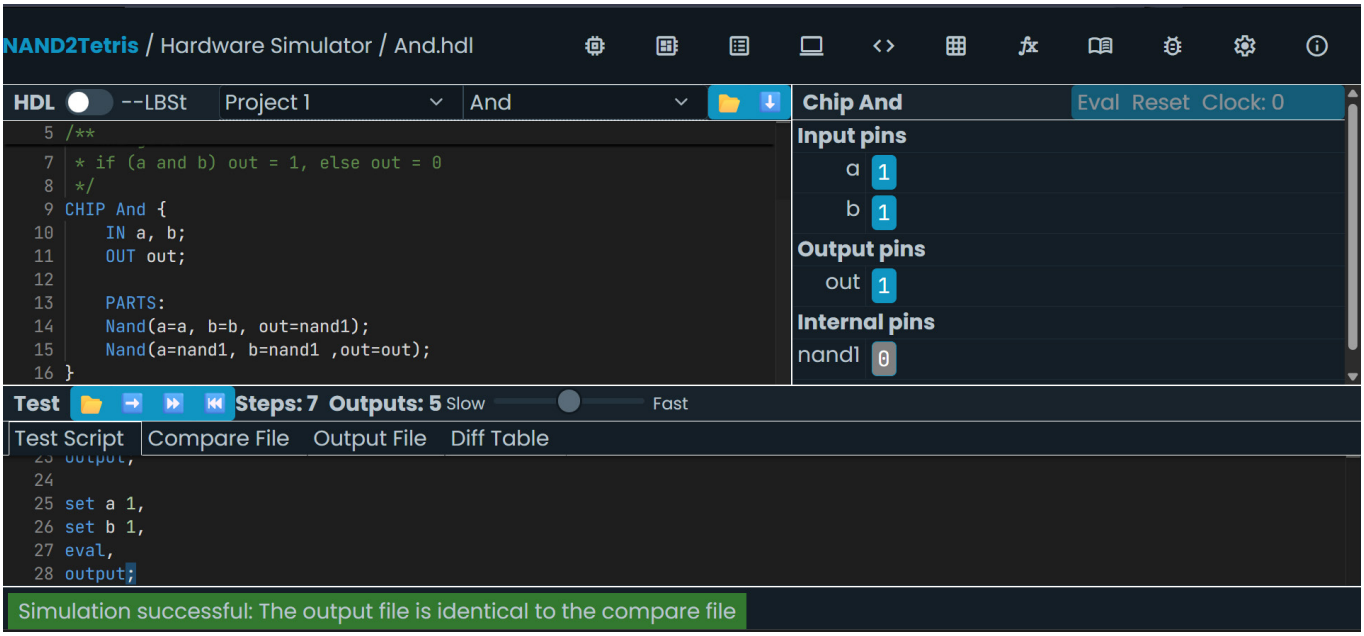
1. And.hdl (Project 1)

Code:

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/And.hdl
/**
 * And gate:
 * if (a and b) out = 1, else out = 0
 */
CHIP And {
    IN a, b;
    OUT out;

    PARTS:
    Nand(a=a, b=b, out=nand1);
    Nand(a=nand1, b=nand1 ,out=out);
}
```

Screenshot (Hardware Simulator output):



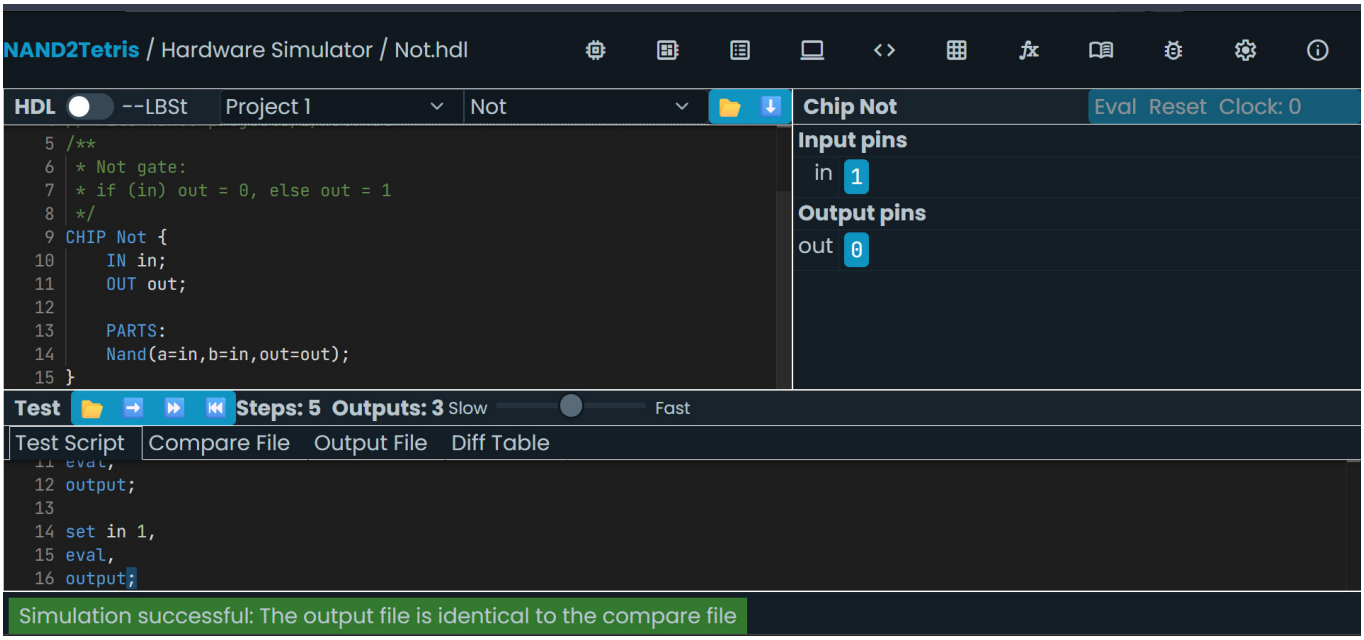
2. Not.hdl (Project 1)

Code:

```
// This file is part of www.nand2tetris.org // and
the book "The Elements of Computing Systems" // by
Nisan and Schocken, MIT Press.
// File name: projects/1/Not.hdl
/**
 * Not gate:
 * if (in) out = 0, else out =
1 */
CHIP Not {
    IN in;
    OUT out;

    PARTS:
    Nand(a=in,b=in,out=out);
}
```

Screenshot (Hardware Simulator output):



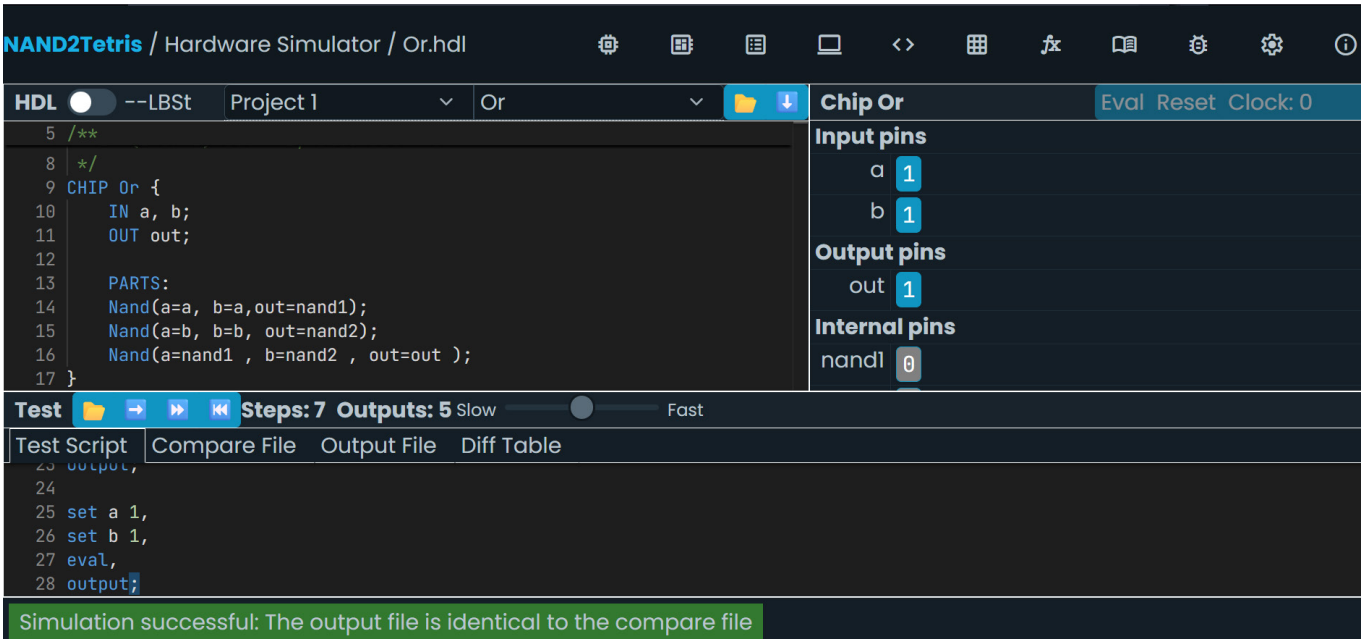
3. Or.hdl (Project 1)

Code:

```
// This file is part of www.nand2tetris.org // and
the book "The Elements of Computing Systems" // by
Nisan and Schocken, MIT Press. // File name:
projects/1/Or.hdl
/**
 * Or gate:
 * if (a or b) out = 1, else out = 0
 */
CHIP Or {
IN a, b;
    OUT out;

    PARTS:
    Nand(a=a, b=a,out=nand1);
    Nand(a=b, b=b, out=nand2);
    Nand(a=nand1 , b=nand2 , out=out );
}
```

Screenshot (Hardware Simulator output):



4. Xor.hdl (Project 1)

Code:

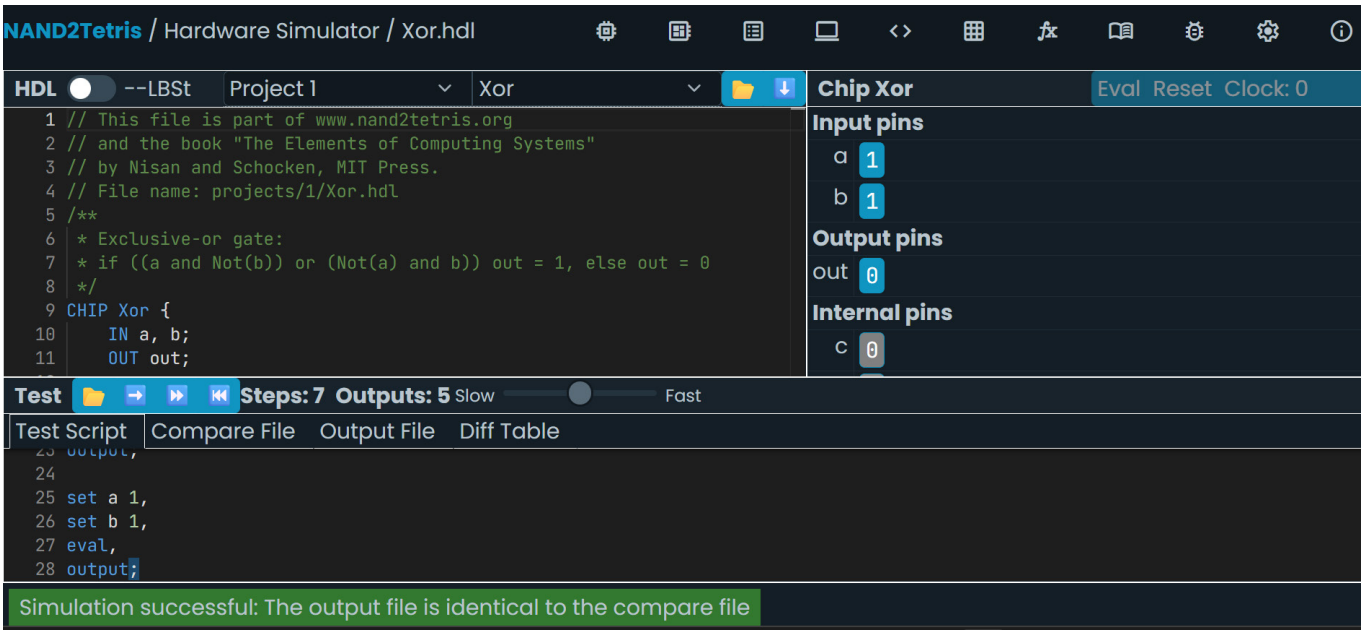
```
// This file is part of www.nand2tetris.org // and the book
"The Elements of Computing Systems" // by Nisan and Schocken,
MIT Press. // File name: projects/1/Xor.hdl /** *
Exclusive-or gate: * if ((a and Not(b)) or (Not(a) and b))
out = 1, else out = 0 */ CHIP Xor {



    IN a, b;
    OUT out;

    PARTS:
    Nand(a=a , b=b , out=c );
    Nand(a=a , b=c , out=d );
Nand(a=c , b=b , out=e );
    Nand(a=d , b=e , out=out );
}
```

Screenshot (Hardware Simulator output):



5. Mux.hdl (Project 1)

Code:

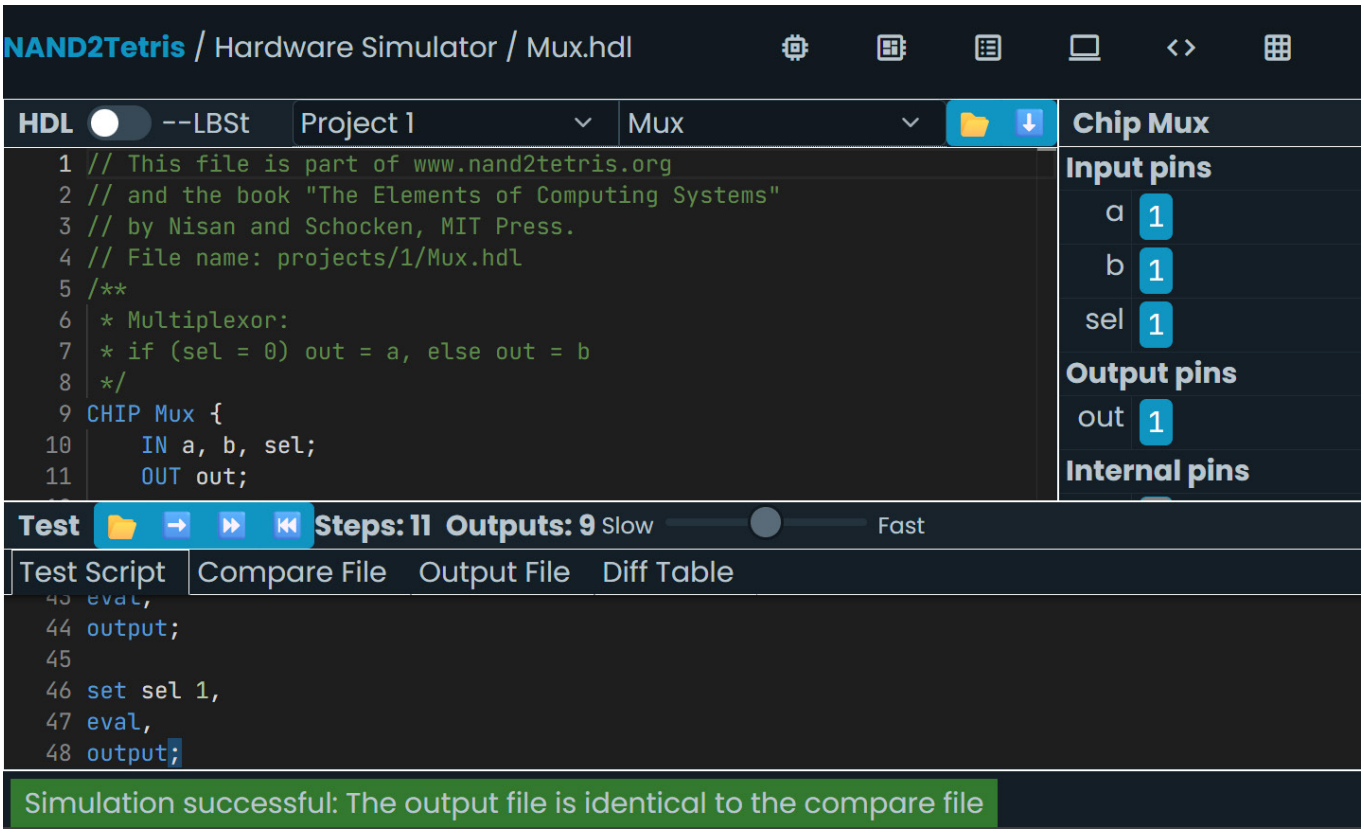
```
// This file is part of www.nand2tetris.org // and
the book "The Elements of Computing Systems" // by
Nisan and Schocken, MIT Press.
// File name: projects/1/Mux.hdl
/**
 * Multiplexor: * if (sel = 0) out =
a, else out = b */

CHIP Mux {
    IN a, b, sel;
    OUT out;

    PARTS:
    Not(in=sel, out=asel);
    And(a=a, b=asel, out=a1);
    And(a=b, b=sel, out=b1);
    Or(a=a1, b=b1, out=out);

}
```

Screenshot (Hardware Simulator output):



6. DMux.hdl (Project 1)

Code:

```
// This file is part of www.nand2tetris.org // and
the book "The Elements of Computing Systems" // by
Nisan and Schocken, MIT Press. // File name:
projects/1/DMux.hdl /**

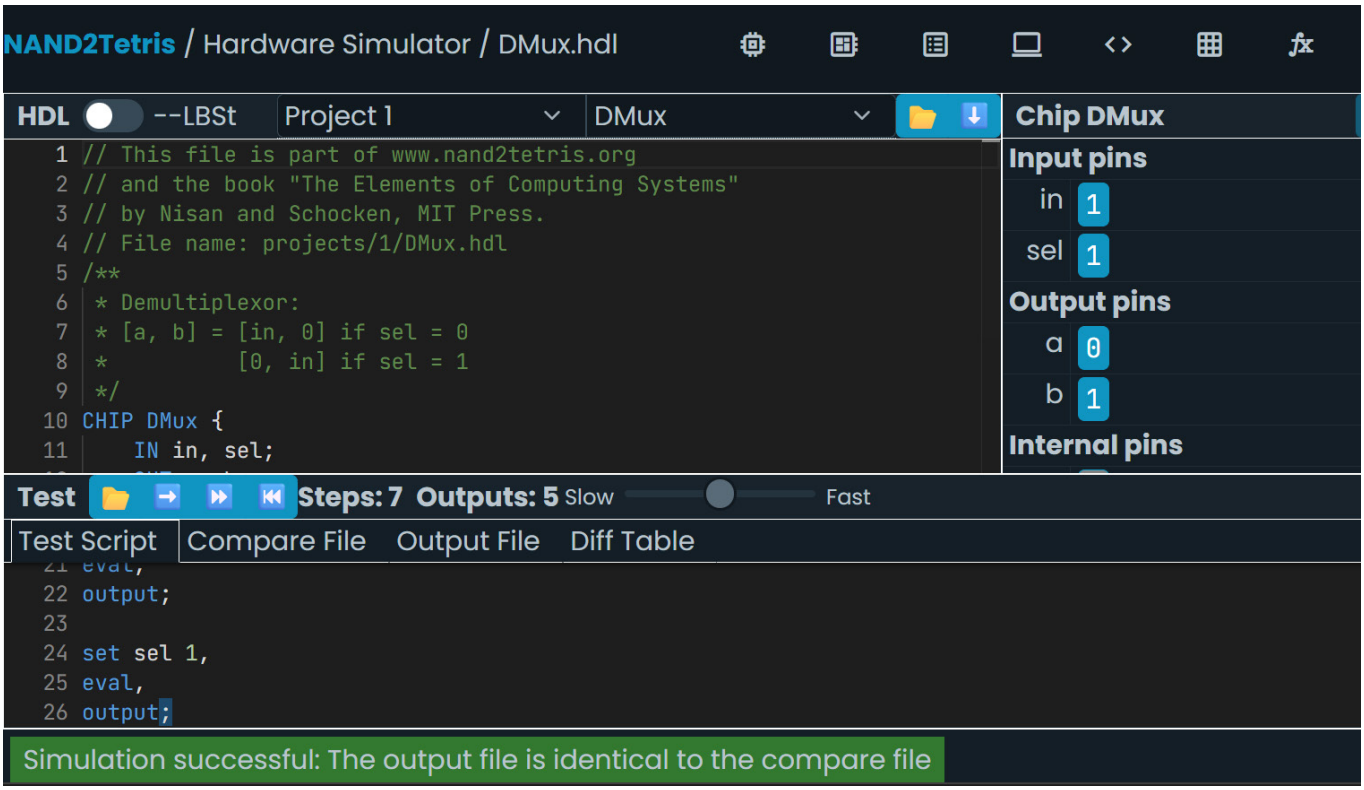
 * Demultiplexor: * [a, b] =

[in, 0] if sel = 0 */
*           [0, in] if sel = 1

CHIP DMux {
    IN in, sel;
    OUT a, b;

    PARTS:    Not(in=sel,
out=nsel);
    And(a=in, b=nsel, out=a);
    And(a=in, b=sel, out=b);
}
```

Screenshot (Hardware Simulator output):



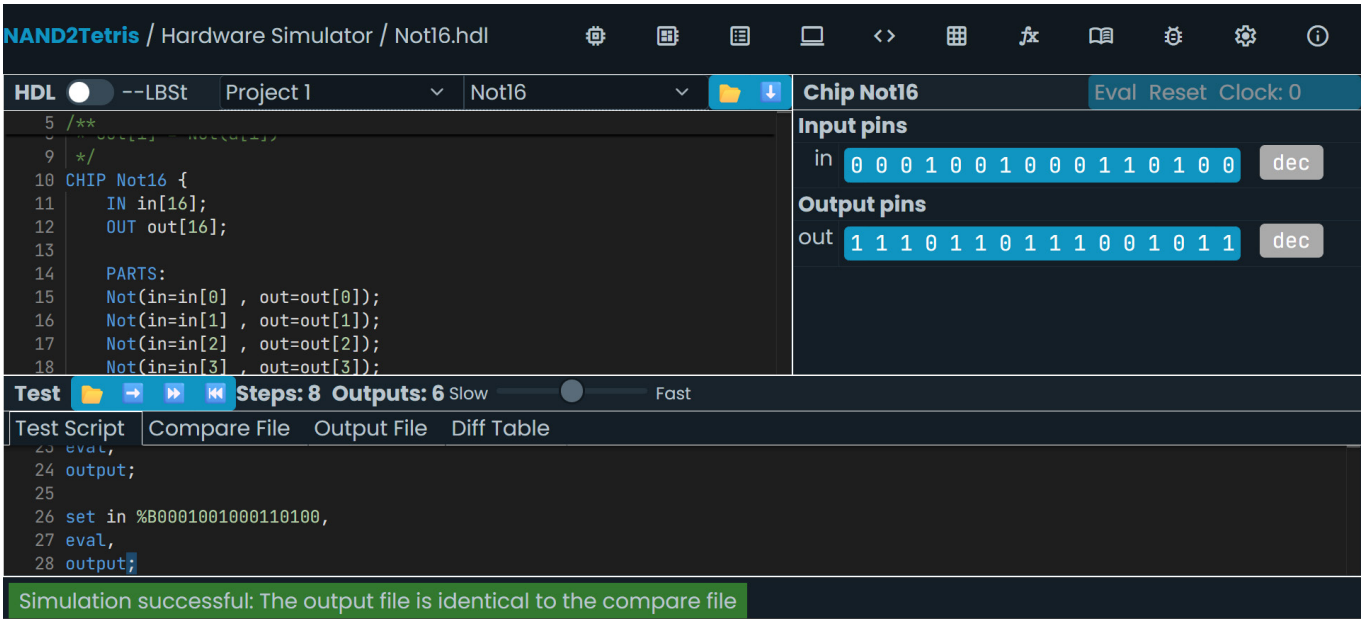
7. Not16.hdl (Project 1)

Code:

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/01/Not16.hdl
/**
 * 16-bit Not gate:
 * for i = 0, ..., 15:
 * out[i] = Not(a[i])
 */
CHIP Not16 {
    IN in[16];
    OUT out[16];

    PARTS:
    Not(in=in[0] , out=out[0]);
    Not(in=in[1] , out=out[1]);
    Not(in=in[2] , out=out[2]);
    Not(in=in[3] , out=out[3]);
    Not(in=in[4] , out=out[4]);
    Not(in=in[5],  out=out[5]);
    Not(in=in[6],  out=out[6]);
    Not(in=in[7],  out=out[7]);
    Not(in=in[8],  out=out[8]);
    Not(in=in[9],  out=out[9]);
    Not(in=in[10], out=out[10]);
    Not(in=in[11], out=out[11]);
    Not(in=in[12], out=out[12]);
    Not(in=in[13], out=out[13]);
    Not(in=in[14], out=out[14]);
    Not(in=in[15], out=out[15]);
}
```

Screenshot (Hardware Simulator output):



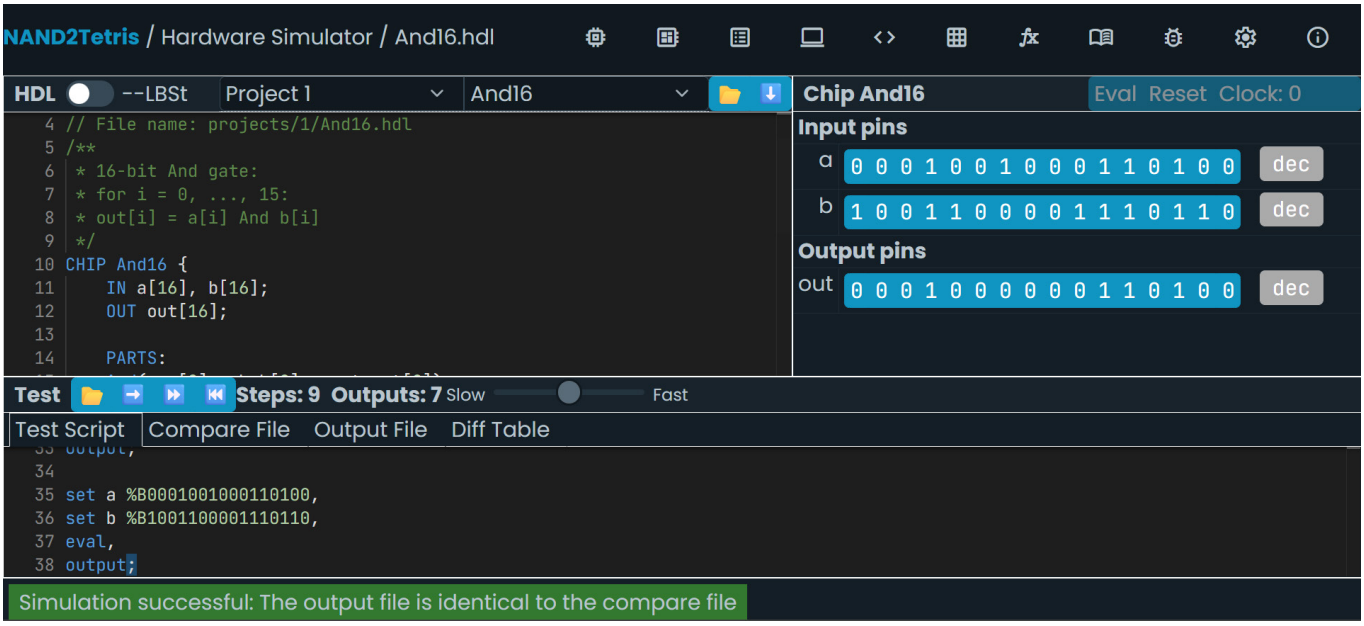
8. And16.hdl (Project 1)

Code:

```
// This file is part of www.nand2tetris.org // and
the book "The Elements of Computing Systems" // by
Nisan and Schocken, MIT Press. // File name:
projects/1/And16.hdl
/**
 * 16-bit And gate:
 * for i = 0, ..., 15:
 * out[i] = a[i] And b[i]
 */
CHIP And16 {
    IN a[16], b[16];
    OUT out[16];

    PARTS:
    And(a=a[0],  b=b[0],  out=out[0]);
    And(a=a[1],  b=b[1],  out=out[1]);
    And(a=a[2],  b=b[2],  out=out[2]);
    And(a=a[3],  b=b[3],  out=out[3]);
    And(a=a[4],  b=b[4],  out=out[4]);
    And(a=a[5],  b=b[5],  out=out[5]);
    And(a=a[6],  b=b[6],  out=out[6]);
And(a=a[7],  b=b[7],  out=out[7]);
    And(a=a[8],  b=b[8],  out=out[8]);
    And(a=a[9],  b=b[9],  out=out[9]);
    And(a=a[10], b=b[10], out=out[10]);
    And(a=a[11], b=b[11], out=out[11]);
    And(a=a[12], b=b[12], out=out[12]);
    And(a=a[13], b=b[13], out=out[13]);
    And(a=a[14], b=b[14], out=out[14]);
    And(a=a[15], b=b[15], out=out[15]);
}
```

Screenshot (Hardware Simulator output):



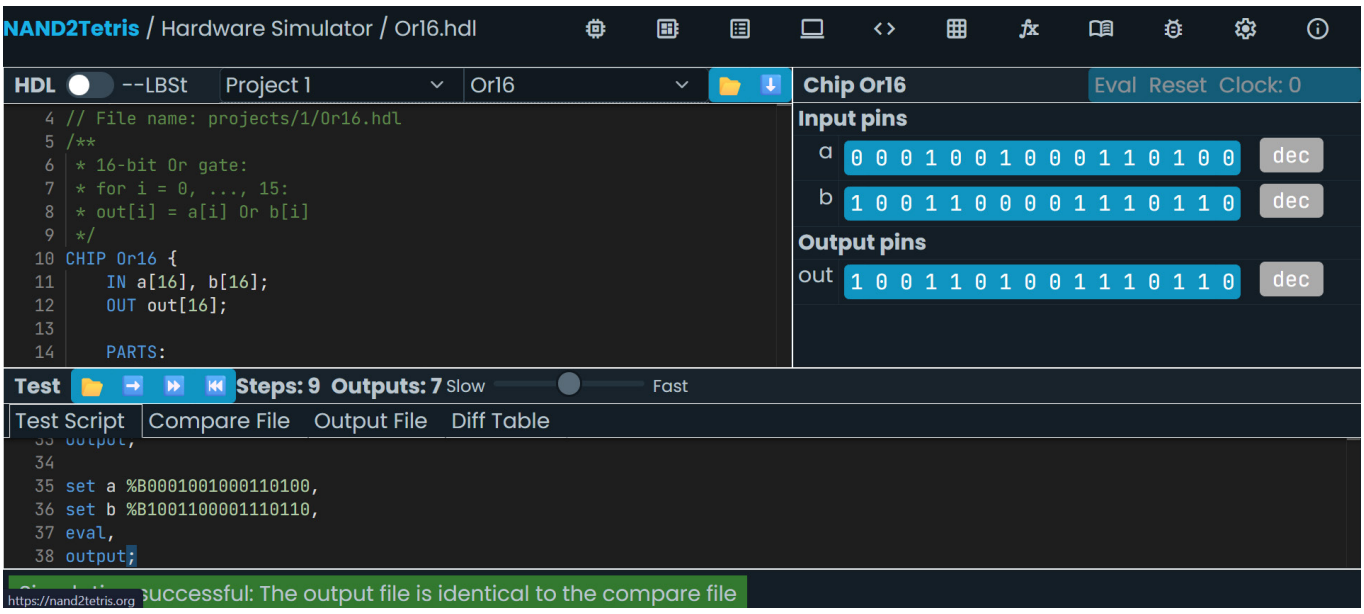
9. Or16.hdl (Project 1)

Code:

```
// This file is part of www.nand2tetris.org // and
the book "The Elements of Computing Systems" // by
Nisan and Schocken, MIT Press. // File name:
projects/1/Or16.hdl
/**
 * 16-bit Or gate:
 * for i = 0, ..., 15:
 * out[i] = a[i] Or b[i]
 */
CHIP Or16 {
    IN a[16], b[16];
    OUT out[16];

    PARTS:
    Or(a=a[0],  b=b[0],  out=out[0]);
    Or(a=a[1],  b=b[1],  out=out[1]);
    Or(a=a[2],  b=b[2],  out=out[2]);
    Or(a=a[3],  b=b[3],  out=out[3]);
    Or(a=a[4],  b=b[4],  out=out[4]);
    Or(a=a[5],  b=b[5],  out=out[5]);
    Or(a=a[6],  b=b[6],  out=out[6]);
    Or(a=a[7],  b=b[7],  out=out[7]);
    Or(a=a[8],  b=b[8],  out=out[8]);
    Or(a=a[9],  b=b[9],  out=out[9]);
    Or(a=a[10], b=b[10], out=out[10]);
    Or(a=a[11], b=b[11], out=out[11]);
    Or(a=a[12], b=b[12], out=out[12]);
    Or(a=a[13], b=b[13], out=out[13]);
    Or(a=a[14], b=b[14], out=out[14]);
    Or(a=a[15], b=b[15], out=out[15]);
}
```

Screenshot (Hardware Simulator output):



10. Mux16.hdl (Project 1)

Code:

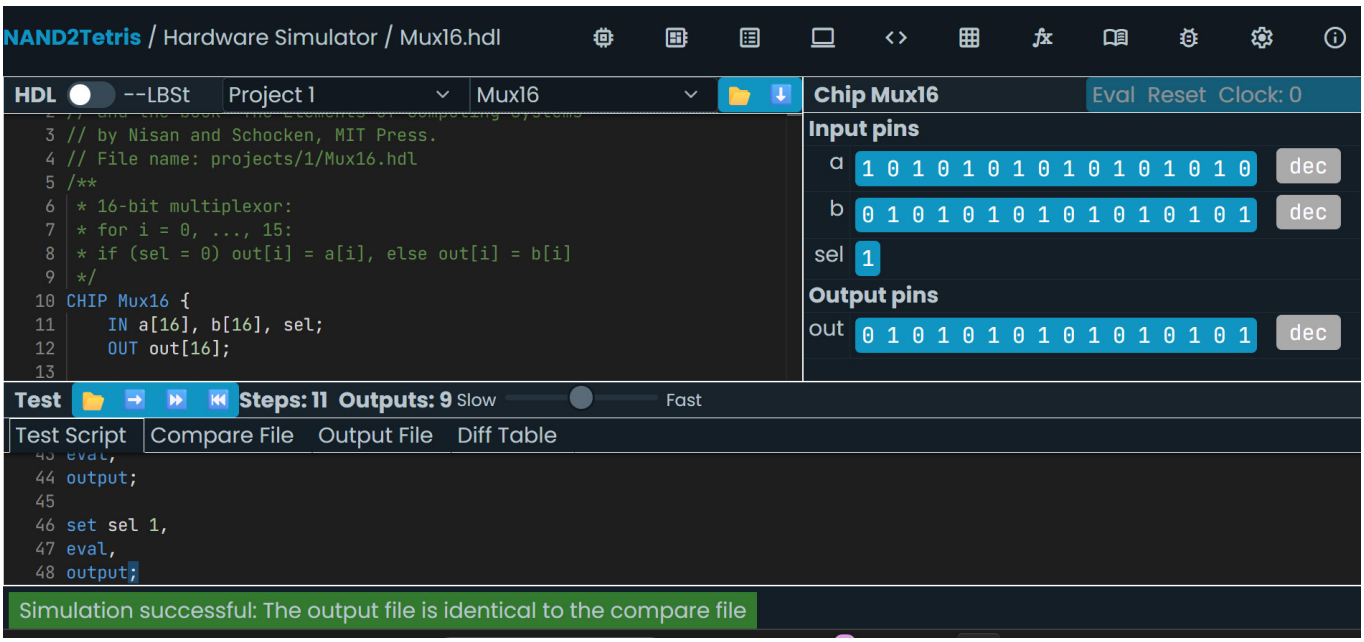
```
// This file is part of www.nand2tetris.org //
and the book "The Elements of Computing Systems" /**
* 16-bit multiplexor:  * for i = 0, ..., 15: * if
 // by Nisan and Schocken, MIT Press.
(sel = 0) out[i] = a[i], else out[i] = b[i] */
// File name: projects/1/Mux16.hdl
OUT out[16];



 CHIP Mux16 {
     IN a[16], b[16], sel;


     PARTS:
     Mux(a=a[0],  b=b[0],  sel=sel, out=out[0]);
     Mux(a=a[1],  b=b[1],  sel=sel, out=out[1]);
     Mux(a=a[2],  b=b[2],  sel=sel, out=out[2]);
     Mux(a=a[3],  b=b[3],  sel=sel, out=out[3]);
     Mux(a=a[4],  b=b[4],  sel=sel, out=out[4]);
     Mux(a=a[5],  b=b[5],  sel=sel, out=out[5]);
     Mux(a=a[6],  b=b[6],  sel=sel, out=out[6]);
     Mux(a=a[7],  b=b[7],  sel=sel, out=out[7]);
     Mux(a=a[8],  b=b[8],  sel=sel, out=out[8]);
     Mux(a=a[9],  b=b[9],  sel=sel, out=out[9]);
     Mux(a=a[10], b=b[10], sel=sel, out=out[10]);
     Mux(a=a[11], b=b[11], sel=sel, out=out[11]);
     Mux(a=a[12], b=b[12], sel=sel, out=out[12]);
     Mux(a=a[13], b=b[13], sel=sel, out=out[13]);
     Mux(a=a[14], b=b[14], sel=sel, out=out[14]);
     Mux(a=a[15], b=b[15], sel=sel, out=out[15]);
 }
```

Screenshot (Hardware Simulator output):



11. Or8Way.hdl (Project 1)

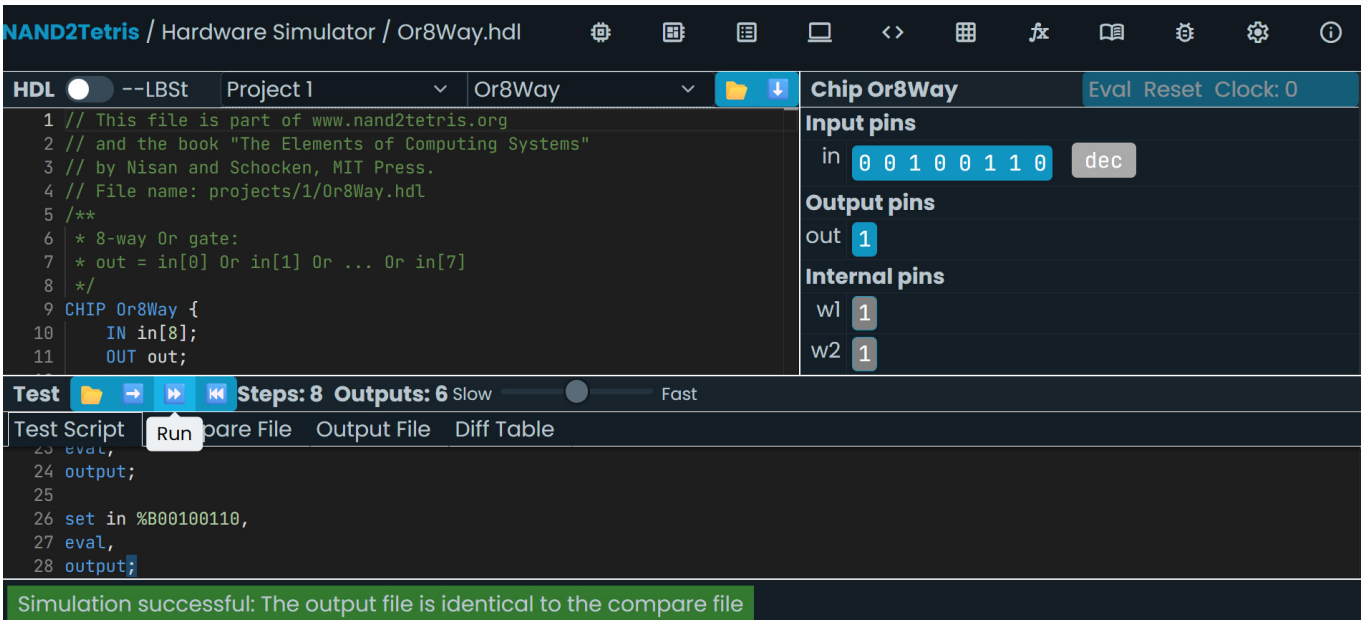
Code:

```
// This file is part of www.nand2tetris.org // and
the book "The Elements of Computing Systems" // by
Nisan and Schocken, MIT Press. // File name:
projects/1/Or8Way.hdl /**

 * 8-way Or gate:  * out = in[0] Or in
[1] Or ... Or in[7] */

CHIP Or8Way {
    IN in
[8];    OUT
out;
    PARTS:    Or(a=in[0], b=in
[1], out=w1);    Or(a=w1, b=in
[2], out=w2);    Or(a=w2, b=in
[3], out=w3);    Or(a=w3, b=in
[4], out=w4);    Or(a=w4, b=in
[5], out=w5);    Or(a=w5, b=in
[6], out=w6);    Or(a=w6, b=in
[7], out=out);
}
```

Screenshot (Hardware Simulator output):



12. Mux4Way16.hdl (Project 1)

Code:

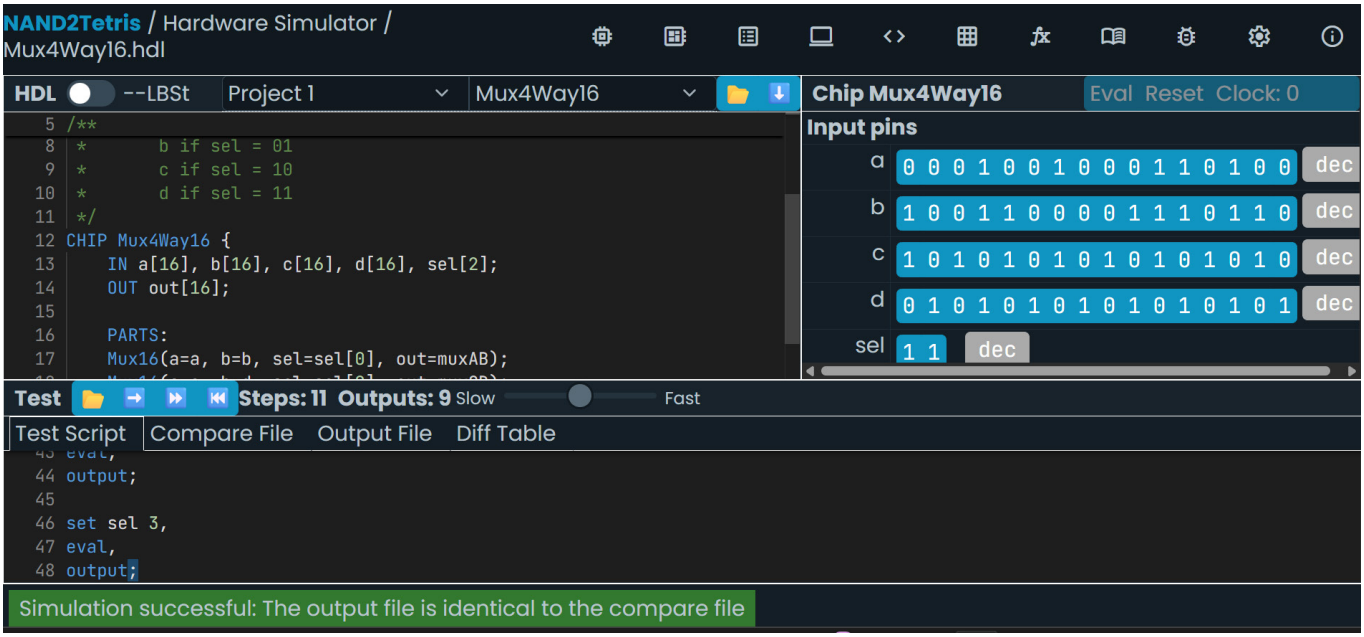
```
// This file is part of www.nand2tetris.org // and
the book "The Elements of Computing Systems" // by
Nisan and Schocken, MIT Press. // File name:
projects/1/Mux4Way16.hdl /**

 * 4-way 16-bit
multiplexor: * out = a if
sel = 00 *   b if sel = 01
*   c if sel = 10 *   d if
sel = 11 */

CHIP Mux4Way16 {
    IN a[16], b[16], c[16], d[16], sel
[2];    OUT out[16];

    PARTS:
    Mux16(a=a, b=b, sel=sel[0], out=muxAB);
    Mux16(a=c, b=d, sel=sel[0], out=muxCD);
    Mux16(a=muxAB, b=muxCD, sel=sel[1], out=out);
}
```

Screenshot (Hardware Simulator output):



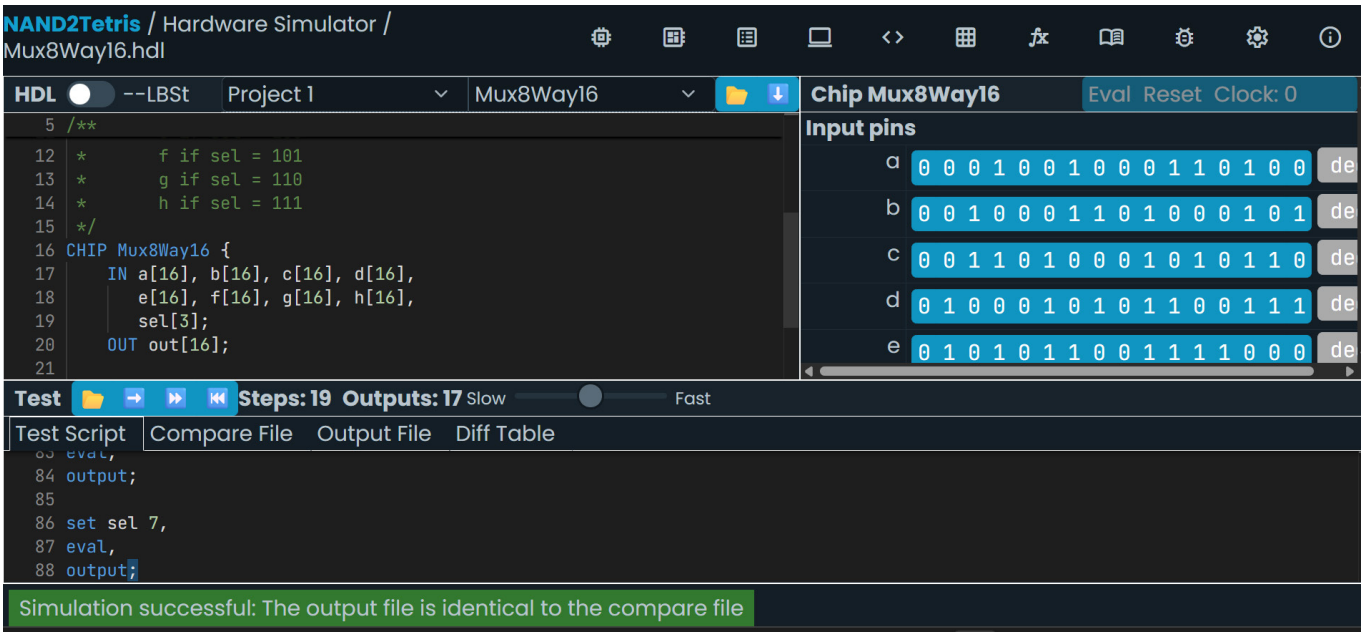
13. Mux8Way16.hdl (Project 1)

Code:

```
// This file is part of www.nand2tetris.org // and
the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Mux8Way16.hdl
/**
 * 8-way 16-bit multiplexor:
 * out = a if sel = 000
 *   b if sel = 001
 *   c if sel = 010
 *   d if sel = 011
 *   e if sel = 100 *
f if sel = 101 *   g if
sel = 110
 *   h if sel = 111
 */
CHIP Mux8Way16 {
    IN a[16], b[16], c[16], d[16],
e[16], f[16], g[16], h[16],
sel[3];
    OUT out[16];

    PARTS:
    Mux4Way16(a=a, b=b, c=c, d=d, sel=sel[0..1], out=outABCD);
    Mux4Way16(a=e, b=f, c=g, d=h, sel=sel[0..1], out=outEFGH);
    Mux16(a=outABCD, b=outEFGH, sel=sel[2], out=out);
}
```

Screenshot (Hardware Simulator output):



14. DMux4Way.hdl (Project 1)

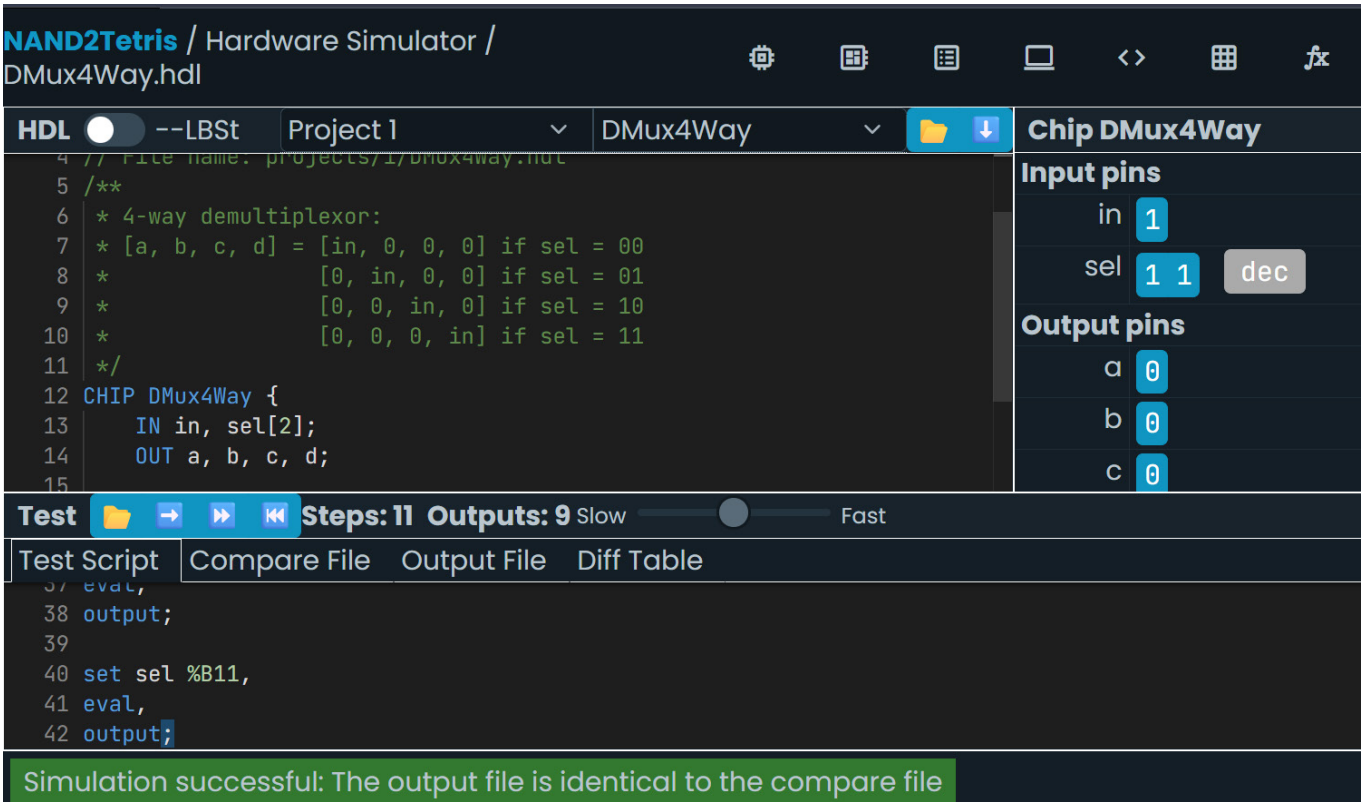
Code:

```
// This file is part of www.nand2tetris.org // and
the book "The Elements of Computing Systems" // by
Nisan and Schocken, MIT Press. // File name:
projects/1/DMux4Way.hdl /**

 * 4-way demultiplexor: * [a, b, c, d] =
[in, 0, 0, 0] if sel = 00
 *                 [0, in, 0, 0] if sel =
*                  01[0, 0, in, 0] if sel =
*                  10[0, 0, 0, in] if sel =
 */ CHIP           11
DMux4Way {
    IN in, sel[2];
OUT a, b, c, d;

    PARTS:
    DMux(in=in, sel=sel[1], a=top, b=bottom);
    DMux(in=top,  sel=sel[0], a=a, b=b);
DMux(in=bottom, sel=sel[0], a=c, b=d);
}
```

Screenshot (Hardware Simulator output):



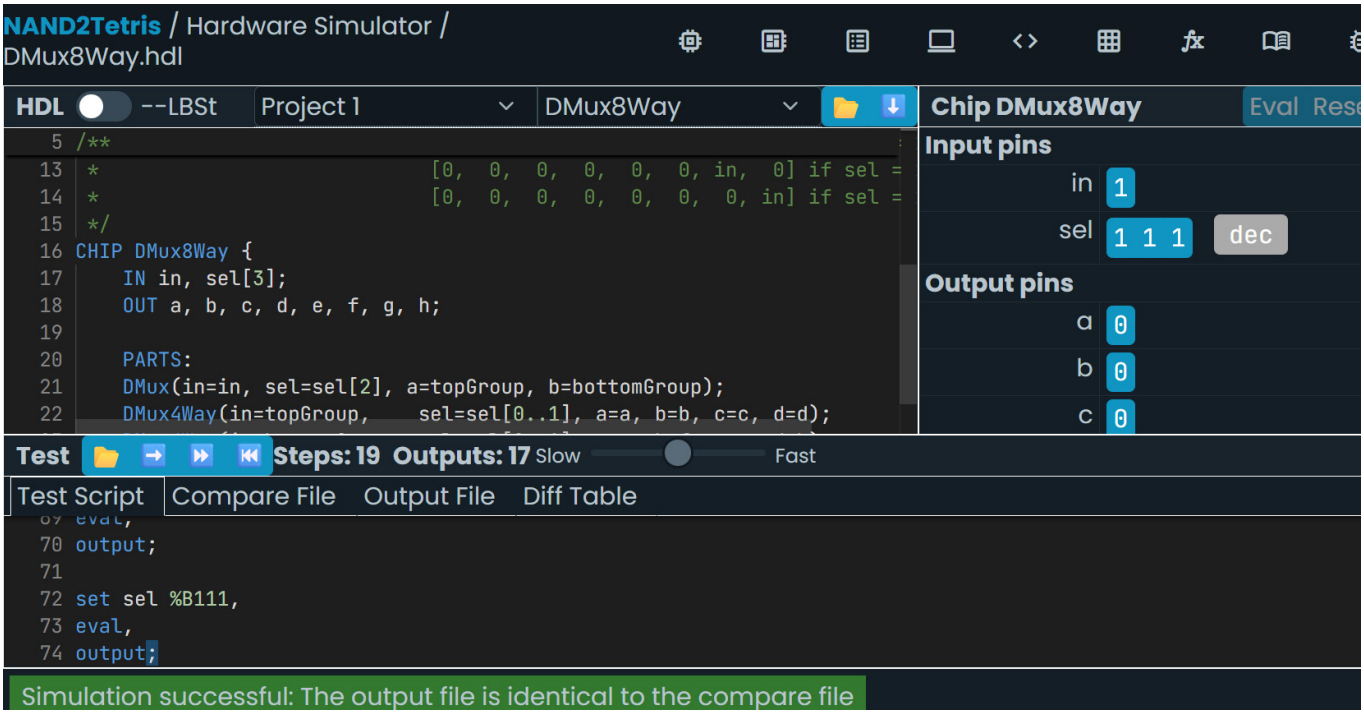
15. DMux8Way.hdl (Project 1)

Code:

```
// This file is part of www.nand2tetris.org // and
the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/DMux8Way.hdl
/** * 8-way
demultiplexor:
 * [a, b, c, d, e, f, g, h] = [in, 0,  0,  0,  0,  0,  0,  0] if sel = 000
 *                            [0, in,  0,  0,  0,  0,  0,  0] if sel = 001
 *                            [0,  0, in,  0,  0,  0,  0,  0] if sel = 010
 *  [0,  0,  0, in,                                  0,  0,  0,  0] if sel = 011
 *                            [0,  0,  0,  0, in,  0,  0,  0] if sel = 100
 *                            [0,  0,  0,  0,  0, in,  0,  0] if sel = 101
 *                            [0,  0,  0,  0,  0,  0, in,  0] if sel = 110
 *                            [0,  0,  0,  0,  0,  0,  0, in] if sel = 111
 */ CHIP DMux8Way
{    IN in, sel
[3];
    OUT a, b, c, d, e, f, g, h;

    PARTS:
    DMux(in=in, sel=sel[2], a=topGroup, b=bottomGroup);
    DMux4Way(in=topGroup,  sel=sel[0..1], a=a, b=b, c=c, d=d);
    DMux4Way(in=bottomGroup, sel=sel[0..1], a=e, b=f, c=g, d=h);
}
```

Screenshot (Hardware Simulator output):



16. HalfAdder.hdl (Project 2)

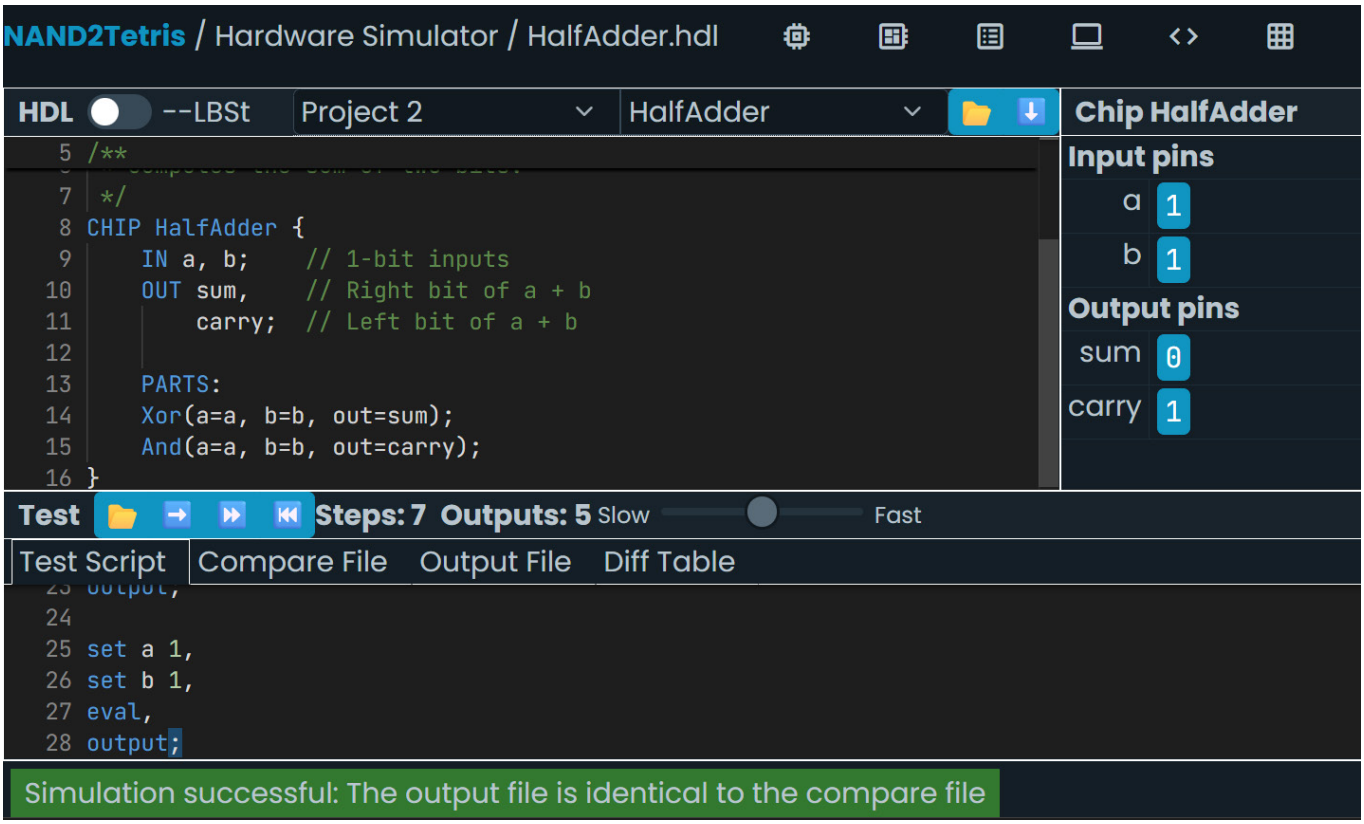
Code:

```
// This file is part of www.nand2tetris.org // and
the book "The Elements of Computing Systems" // by
Nisan and Schocken, MIT Press. // File name:
projects/2/HalfAdder.hdl /**

 * Computes the sum of two bits.
 */ CHIP HalfAdder {    IN a, b;  //
1-bit inputs    OUT sum,  // Right
bit of a + b         carry;  // Left
bit of a + b


    PARTS:
    Xor(a=a, b=b, out=sum);
    And(a=a, b=b, out=carry);
}
```

Screenshot (Hardware Simulator output):



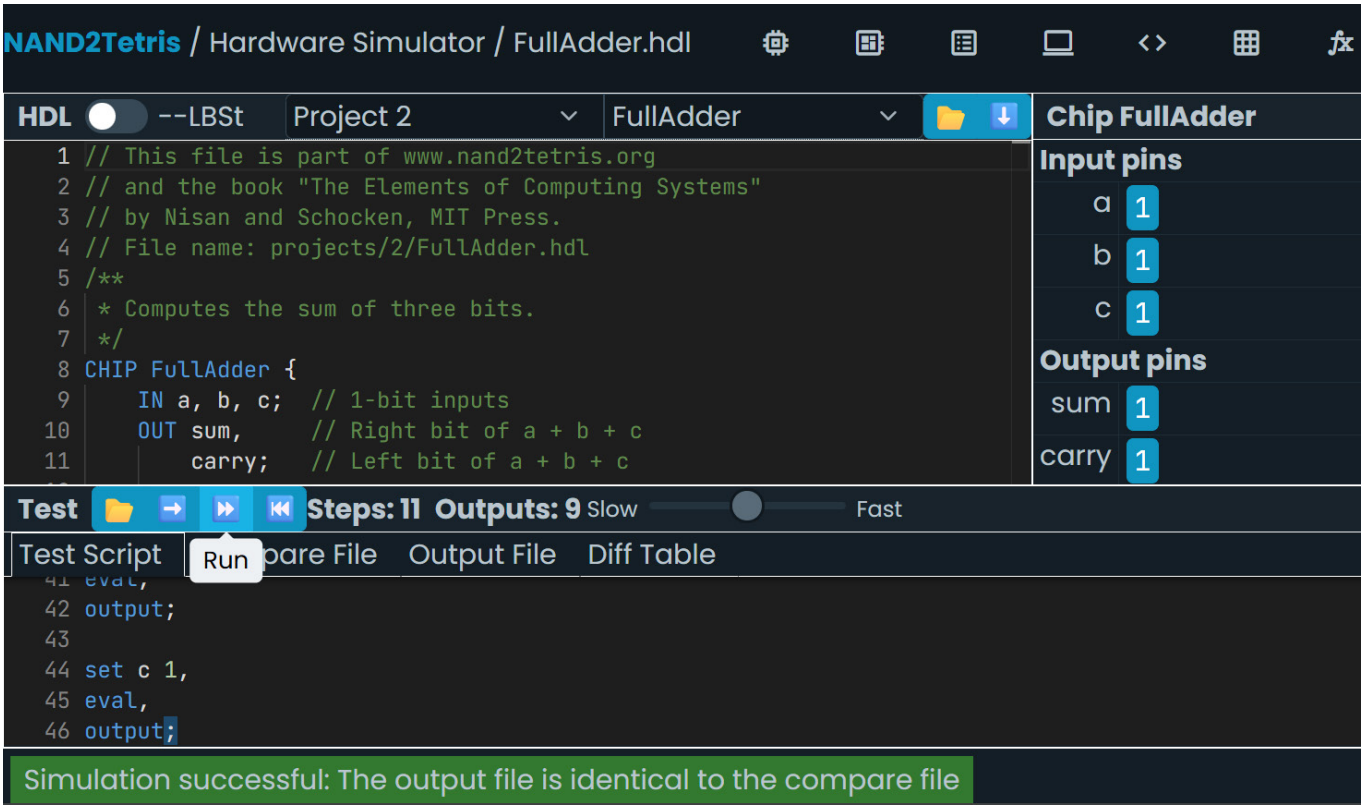
17. FullAdder.hdl (Project 2)

Code:

```
// This file is part of www.nand2tetris.org // and
the book "The Elements of Computing Systems" // by
Nisan and Schocken, MIT Press. // File name:
projects/2/FullAdder.hdl
/**
 * Computes the sum of three bits.
 */ CHIP FullAdder {    IN a, b, c;  //
1-bit inputs    OUT sum,   // Right bit
of a + b + c        carry;   // Left bit
of a + b + c


    PARTS:
    HalfAdder(a=a, b=b, sum=s1, carry=c1);
    HalfAdder(a=s1, b=c, sum=sum, carry=c2);
    Or(a=c1, b=c2, out=carry);
}
```

Screenshot (Hardware Simulator output):



18. Add16.hdl (Project 2)

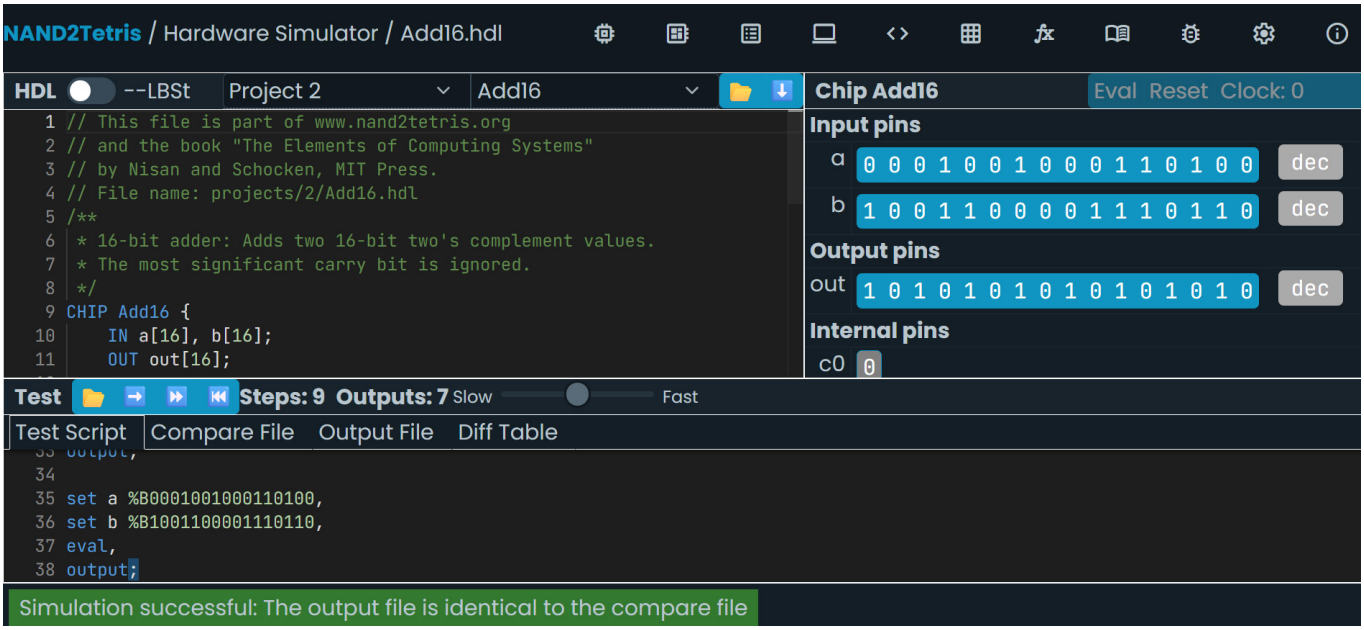
Code:

```
// This file is part of www.nand2tetris.org // and
the book "The Elements of Computing Systems" // by
Nisan and Schocken, MIT Press.
// File name: projects/2/Add16.hdl
/**
*  16-bit adder: Adds two 16-bit two's complement values.
*  The most significant carry bit is ignored.
 */
CHIP Add16 {
    IN a[16], b[16];
    OUT out[16];

    PARTS:
    HalfAdder(a=a[0], b=b[0] , sum=out[0] , carry=c0 );
    FullAdder(a=a[1] , b=b[1] , c=c0 , sum=out[1] , carry=c1 );
    FullAdder(a=a[2] , b=b[2] , c=c1 , sum=out[2] , carry=c2);
    FullAdder(a=a[3] , b=b[3] , c=c2 , sum=out[3] , carry=c3 );
    FullAdder(a=a[4] , b=b[4] , c=c3 , sum=out[4] , carry=c4 );
    FullAdder(a=a[5] , b=b[5] , c=c4 , sum=out[5] , carry=c5 );
    FullAdder(a=a[6] , b=b[6] , c=c5 , sum=out[6] , carry=c6 );
    FullAdder(a=a[7] , b=b[7] , c=c6 , sum=out[7] , carry=c7 );
    FullAdder(a=a[8] , b=b[8] , c=c7 , sum=out[8] , carry=c8 );
    FullAdder(a=a[9] , b=b[9] , c=c8 , sum=out[9] , carry=c9);
    FullAdder(a=a[10] , b=b[10] , c=c9 , sum=out[10] , carry=c10 );
    FullAdder(a=a[11] , b=b[11] , c=c10 , sum=out[11] , carry=c11 );
    FullAdder(a=a[12] , b=b[12] , c=c11, sum=out[12] , carry=c12 );
    FullAdder(a=a[13] , b=b[13] , c=c12, sum=out[13] , carry=c13 );
    FullAdder(a=a[14] , b=b[14] , c=c13, sum=out[14] , carry=c14 );
    FullAdder(a=a[15] , b=b[15] , c=c14, sum=out[15] , carry=c15 );


}
```

Screenshot (Hardware Simulator output):



19. Inc16.hdl (Project 2)

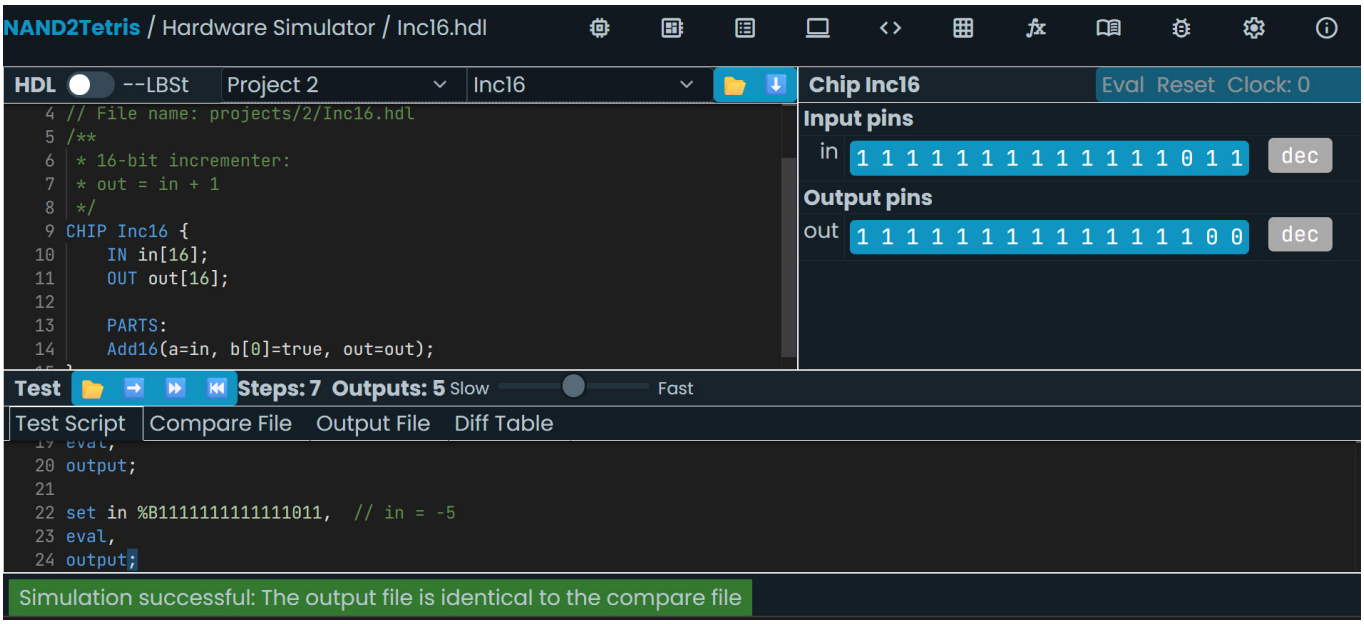
Code:

```
// This file is part of www.nand2tetris.org // and
the book "The Elements of Computing Systems" // by
Nisan and Schocken, MIT Press. // File name:
projects/2/Inc16.hdl /**

 * 16-bit
incrementer: * out =
in + 1 */
CHIP Inc16 {
    IN in[16];
OUT out[16];

    PARTS:
    Add16(a=in, b[0]=true, out=out);
}
```

Screenshot (Hardware Simulator output):



20. ALU.hdl (Project 2)

Code:

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/2/ALU.hdl /**
* ALU (Arithmetic Logic Unit):

 * Computes out = one of the following functions:
 *                0, 1, -1,x, y, !x, !y, -x,
*                 -y,x + 1, y + 1, x - 1, y
*                 - 1,x + y, x - y, y - x,x
*                 & y, x | y
*
 * on the 16-bit inputs x, y, * according to the
input bits zx, nx, zy, ny, f, no. * In addition,
computes the two output bits: * if (out == 0) zr =
1, else zr = 0 * if (out < 0)  ng = 1, else ng = 0
*/

// Implementation: Manipulates the x and y inputs
// and operates on the resulting values, as follows:
// if (zx == 1) sets x = 0  // 16-bit constant
// if (nx == 1) sets x = !x       // bitwise not //
// if (zy == 1) sets y = 0        16-bit constant //
// if (ny == 1) sets y = !y       bitwise not
// if (f == 1)  sets out = x + y  // integer 2's complement addition
// if (f == 0)  sets out = x & y  // bitwise and
// if (no == 1) sets out = !out   // bitwise not

CHIP ALU {
    IN
        x[16], y[16],  // 16-bit inputs
zx, // zero the x input?        nx, // negate the x
input?        zy, // zero the y input?        ny, //
negate the y input?        f,  // compute (out = x +
y) or (out = x & y)?        no; // negate the out
output?

    OUT
        out[16], // 16-bit output        zr,  //
if (out == 0) equals 1, else 0        ng;  // if
(out < 0)  equals 1, else 0

    PARTS:
    // x zero / negate
      Mux16(a=x, sel=zx, out=x1);
      Not16(in=x1, out=negatedX);
      Mux16(a=x1, b=negatedX, sel=nx, out=x2);

      // y zero / negate
      Mux16(a=y, sel=zy, out=y1);
      Not16(in=y1, out=negatedY);
      Mux16(a=y1, b=negatedY, sel=ny, out=y2);

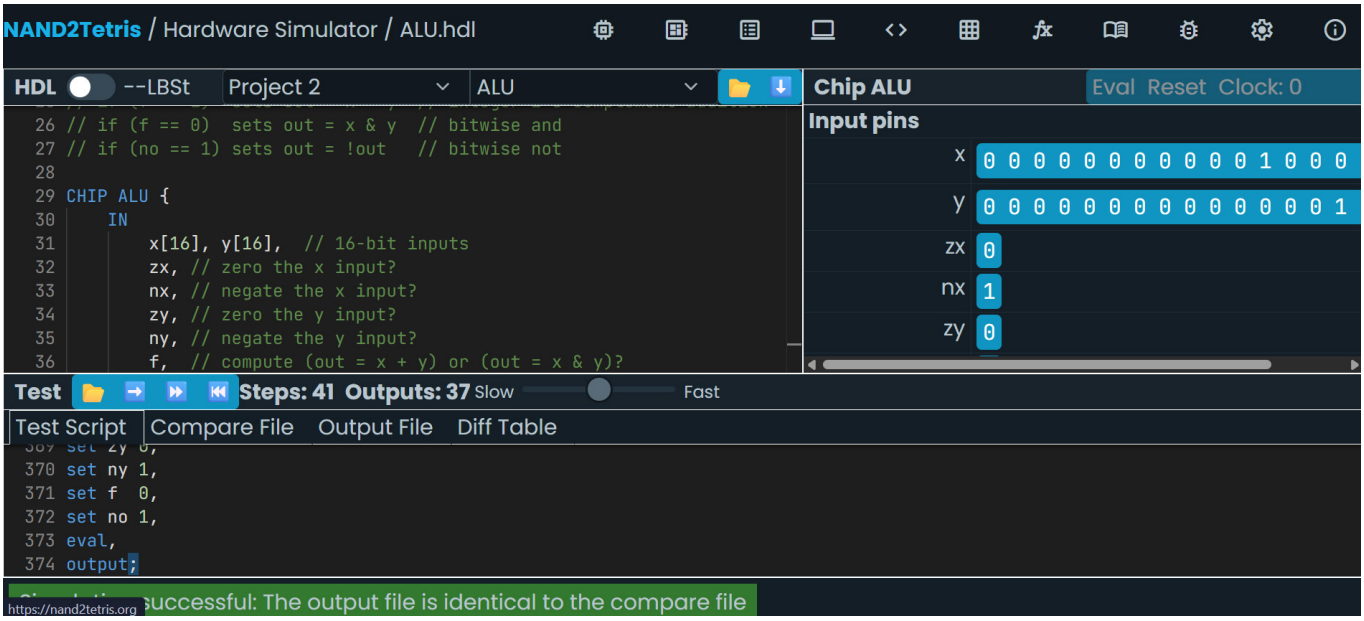
      // x + y OR x & y
      Add16(a=x2, b=y2, out=added);
      And16(a=x2, b=y2, out=xyAnd);
      Mux16(a=xyAnd, b=added, sel=f, out=result);

      // negate the output?
      Not16(in=result, out=negatedResult);
      Mux16(a=result, b=negatedResult, sel=no, out=out, out[15]=firstOut, out[0..7]=finalLeft, out[8..15]=finalRight);

      // output == 0 (zr)
      Or8Way(in=finalLeft, out=zrl);
      Or8Way(in=finalRight, out=zrr);
Or(a=zrl, b=zrr, out=nzr);
      Not(in=nzr, out=zr);

      // output < 0 (ng)
      And(a=firstOut, b=true, out=ng);
}
```

Screenshot (Hardware Simulator output):



Thank You!